

Topics Covered

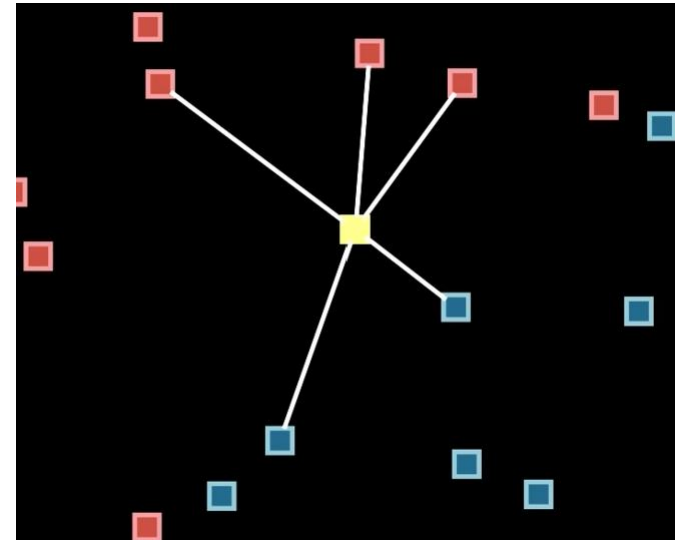
- K-Nearest Neighbors (KNN)
- How KNN works
- Distance Metrics in KNN
- Choosing the value of K
- Code Implementation
- Summary

K-Nearest Neighbors (KNN)

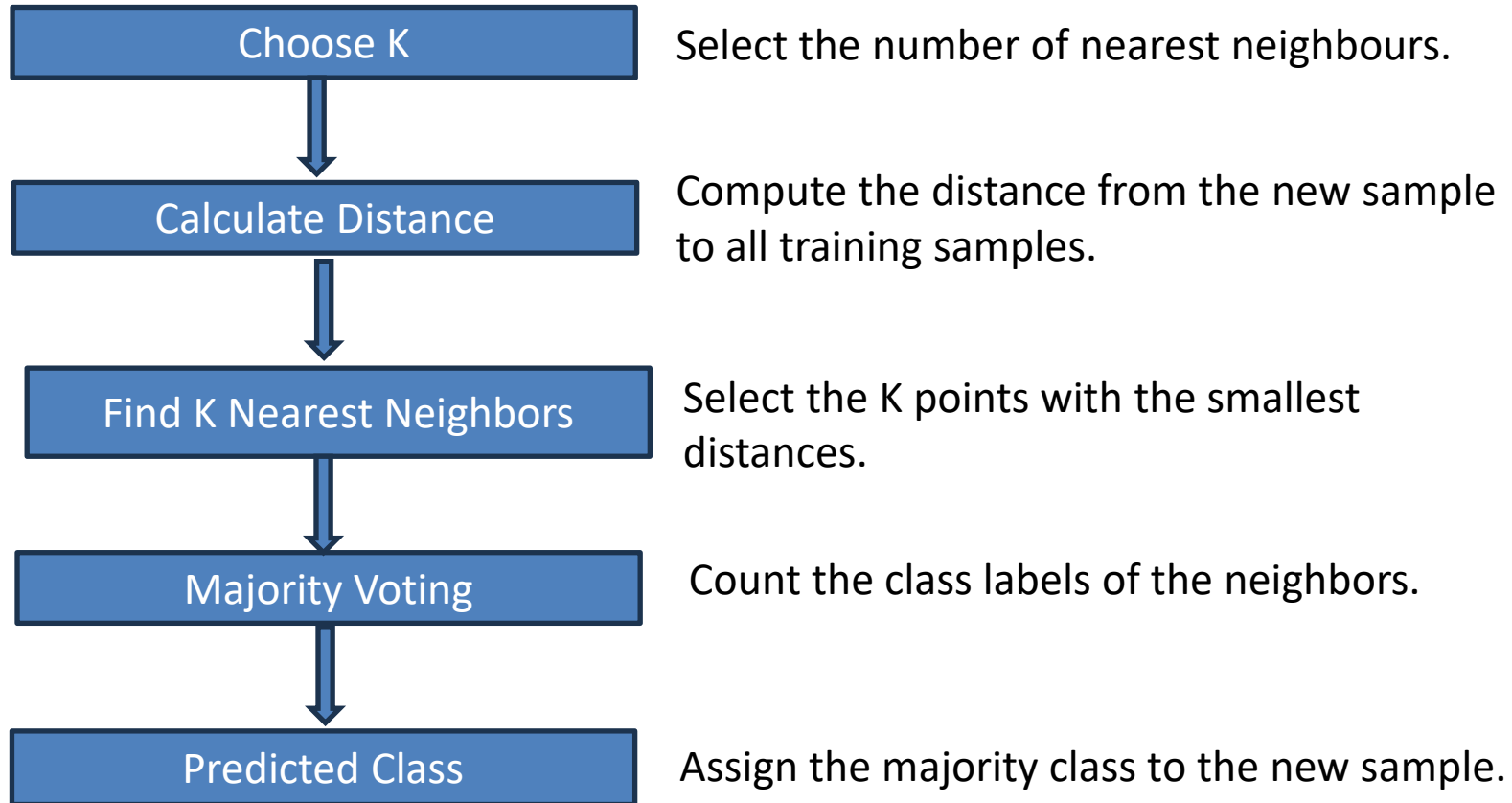
- ❑ K-Nearest Neighbors (KNN) is a supervised machine learning algorithm that predicts the class of a new data point based on the majority class of its K nearest neighbors.

Key Characteristics:

- Training phase: Stores the training data (no explicit model is built).
- Prediction phase: Predicts the class using the K nearest neighbors.
- Simple and easy to understand and implement.



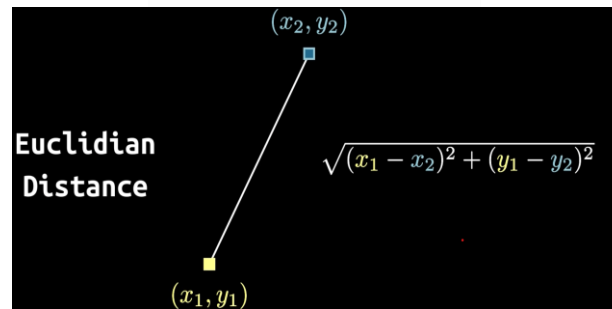
How KNN works



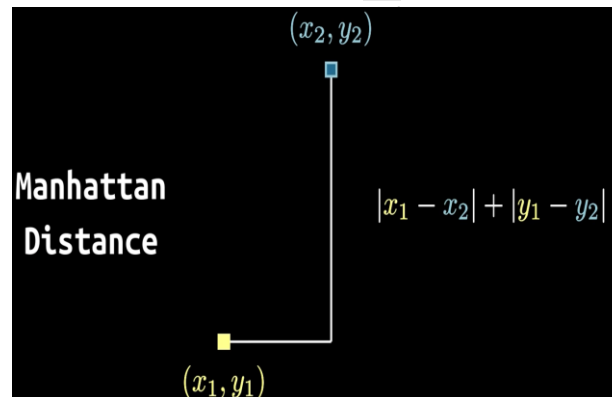
Distance Metrics in KNN

- ❑ **Euclidean distance** measures the straight-line distance between two data points.

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



- ❑ **Manhattan distance** measures the distance when movement is restricted to horizontal and vertical paths (similar to city blocks).



Choosing the Value of K

- ❑ K is the hyperparameter that determines the number of nearest neighbors used for prediction.
- Small K (e.g., $K=1$): Captures fine details but is highly sensitive to noise and outliers.
- Large K: Smooths out predictions and makes the model more stable. However, if K is too large, it can include points from unrelated categories, causing underfitting.
- Rule of Thumb: It is standard practice to pick an odd number for K to prevent voting ties

Code Implementation

```
16 import numpy as np
17 import pandas as pd
18 from sklearn.model_selection import train_test_split
19 from sklearn.metrics import confusion_matrix, accuracy_score, precision_score
20
21
22 filename = "glass.csv"
23 df = pd.read_csv(filename)
24 print("Dataset used      :", filename)
25 print("Columns          :", list(df.columns)) # verify which column is the label
26 print(df.head(), "\n")
27
28 label = df.columns[-1] # convention: last column = class label
29 X = df.drop(label, axis=1).to_numpy() # features -> NumPy array
30 y = df[label].to_numpy() # labels -> NumPy array
31
32 n, d = X.shape
33 print("n (rows/examples) :", n)
34 print("d (features)       :", d, "\n")
35
36 X_train, X_test, y_train, y_test = train_test_split(
37     X, y, test_size=0.30, random_state=42
38 )
39 print("X_train shape      :", X_train.shape)
40 print("X_test shape       :", X_test.shape, "\n")
41
42 n_train = X_train.shape[0] # theoretical max value k could take is n_train
43
44 # -----
45 # kNN FROM SCRATCH
46 # -----
47 def predict_one(x_query, X_train, y_train, k):
48     """Predict the label of a single point x_query."""
49     # Euclidean distance from x_query to EVERY training point.
50     # (X_train - x_query) subtracts x_query from each row (broadcasting),
51     # we square, sum across the feature axis, then take the square root.
52     distances = np.sqrt(np.sum((X_train - x_query) ** 2, axis=1))
```

No plots to show

Run plot-generating code in the Editor or IPython console to see your figures appear here. This pane

Help Variable Explorer Debugger Profiler Plots Files

Console 1/A x

```
[ 1  1  0  0  0  0]
Accuracy      : 0.5846
Precision (macro) : 0.4976

=====
k = 15
=====
Confusion matrix (rows = actual, cols = predicted):
[[17  2  0  0  0  0]
 [10 11  1  0  0  1]
 [ 3  1  0  0  0  0]
 [ 0  4  0  2  0  0]
 [ 1  0  0  0  0  2]
 [ 1  1  0  0  0  8]]
Accuracy      : 0.5846
Precision (macro) : 0.4729

=====
SUMMARY
=====

   k | accuracy | precision
-----|-----|-----
  1 |    0.7692 |    0.7164
  2 |    0.6615 |    0.6342
  3 |    0.6462 |    0.6364
  5 |    0.5846 |    0.3753
 10 |    0.5846 |    0.4976
 15 |    0.5846 |    0.4729
-----|-----|-----
Best accuracy : k = 1 (accuracy = 0.7692)
Best precision: k = 1 (precision = 0.7164)
```

Summary

❑ Results and Discussion

- **Banana dataset**

- Large dataset with two standardized features.
- Performance improved as K increased.
- Best performance: K = 10 (90.19% accuracy).

- **Glass dataset**

- Small dataset with six classes.
- Larger K values introduced neighbors from different classes.
- Best performance: K = 1 (76.92% accuracy).

- **Wine dataset**

- Small dataset with thirteen features and three classes.
- Without feature scaling, large-range features influenced Euclidean distance.
- Best performance: K = 1 (79.63% accuracy).

❑ Overall Conclusion

- The optimal value of K depends on the characteristics of the dataset.
- Testing multiple values of K is important because there is no universally best choice of K.